17 November 2026

Struktur Data

Tree Traversal

Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

Fakultas Teknik Industri

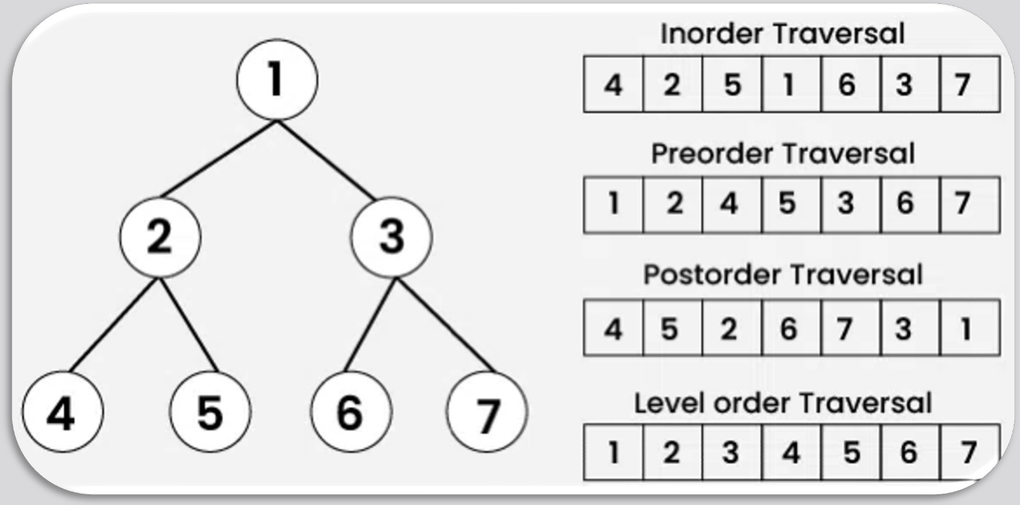
Universitas Pembangunan Nasional “Veteran” Yogyakarta



Konsep Tree Traversal

Tree traversal adalah **proses sistematis mengunjungi setiap node** dalam **struktur pohon** untuk membaca, memproses, atau memanipulasi data yang tersimpan di dalamnya.

Proses ini berperan penting dalam **pencarian**, **pengurutan**, **evaluasi ekspresi**, dan **pemrosesan struktur hierarki**, yang dapat dilakukan secara **rekursif** atau iteratif tergantung pada jenis pohon dan tujuan pemrosesan.



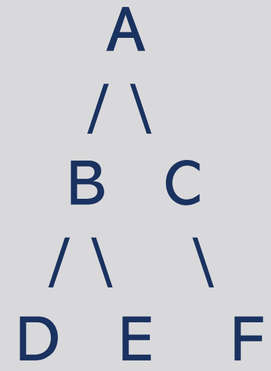
Istilah Penting Tree Traversal

Istilah dasar terkait **tree**:

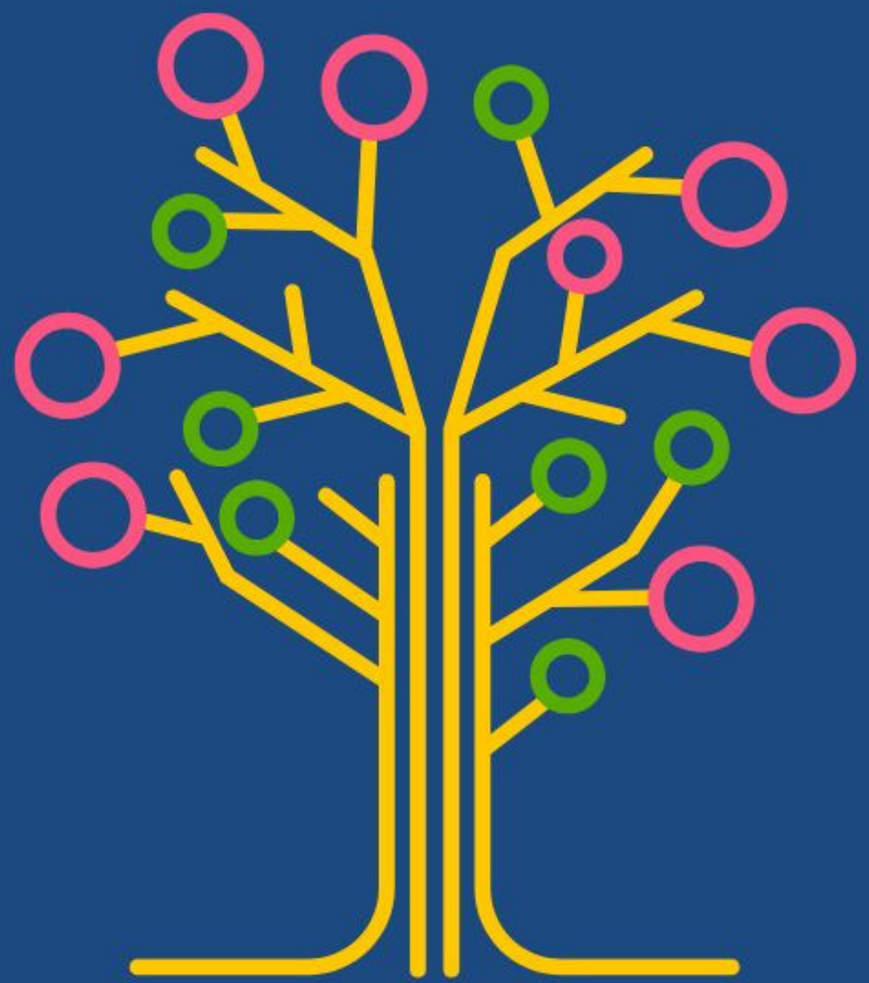
- ✓ **Node**: Elemen dasar dalam tree yang menyimpan data.
- ✓ **Root**: Node paling atas dalam tree.
- ✓ **Leaf Node**: Node yang tidak memiliki anak.
- ✓ **Subtree**: Struktur pohon di dalam pohon yang bersangkutan.
- ✓ **Height & Depth**: Height adalah panjang jalur terpanjang dari node ke leaf, sedangkan depth adalah panjang jalur dari root ke node tersebut.

Istilah Penting Tree Traversal

Terdapat beberapa **metode traversal** yang umum digunakan, masing-masing memiliki **tujuan** & **cara berbeda**. Sebagai ilustrasi, misalkan pohon berikut:



- ✓ **Preorder**: A B D E C F
- ✓ **Inorder**: D B E A C F
- ✓ **Postorder**: D E B F C A
- ✓ **Level-order**: A B C D E F



Kategori Utama
Tree Traversal

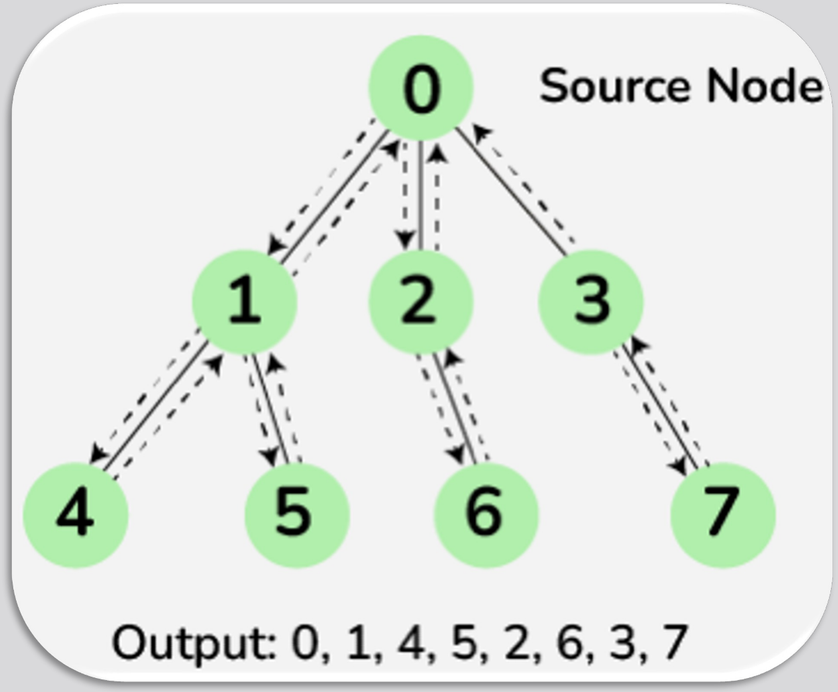
1. Depth-First Traversal (DFT)

2. Breadth-First Traversal (BFT)

Depth-First Traversal

Traversal yang berfokus pada **penelusuran cabang pohon** sedalam mungkin sebelum beralih ke cabang lainnya. **DFT** biasanya **diimplementasikan secara rekursif**, meskipun dapat juga menggunakan **stack**. Jenis-jenis DFT antara lain:

- ✓ **Preorder Traversal (Root – Left – Right)**: Node akar dikunjungi terlebih dahulu, dilanjutkan dengan subtree kiri, dan terakhir subtree kanan. Cocok untuk membuat salinan pohon atau evaluasi ekspresi dalam bentuk prefix. Contoh urutan: A, B, D, E, C, F.



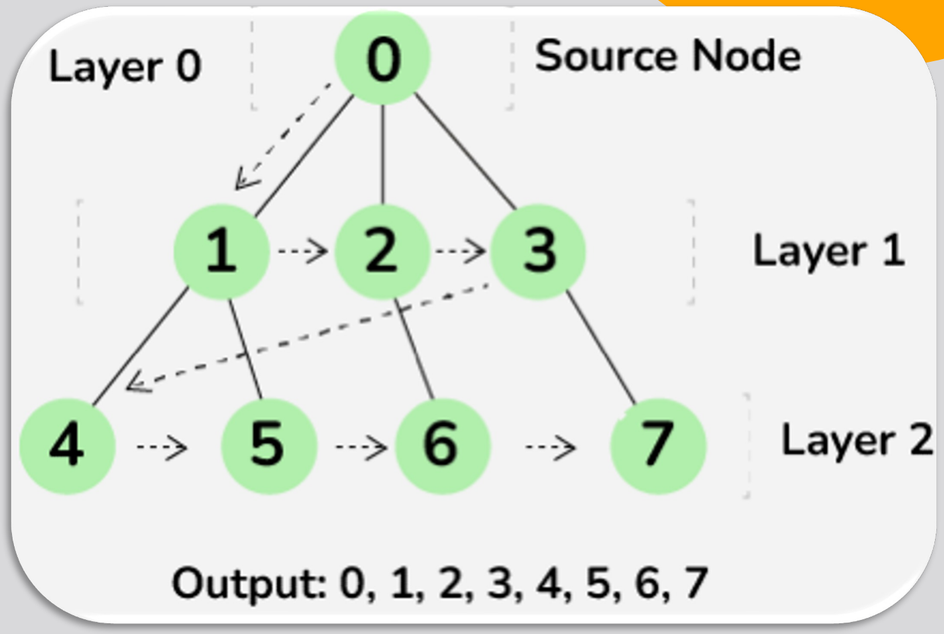
Depth-First Traversal

- ✓ **Inorder Traversal (Left – Root – Right)**: Subtree kiri dikunjungi lebih dahulu, kemudian node akar, dan terakhir subtree kanan. Pada Binary Search Tree (BST), traversal ini menghasilkan data dalam urutan terurut (ascending), sehingga sering digunakan dalam pengurutan dan pencarian data.

- ✓ **Postorder Traversal (Left – Right – Root)**: Subtree kiri dikunjungi lebih dahulu, lalu subtree kanan, dan terakhir node akar. Digunakan dalam operasi yang memerlukan pemrosesan anak sebelum induk, seperti penghapusan pohon atau evaluasi ekspresi postfix.

Breadth-First Traversal

Traversal ini **mengunjungi node** berdasarkan tingkatannya (level), dimulai dari **node akar (level 0)**, kemudian seluruh node di level berikutnya dari **kiri ke kanan**. Umumnya diimplementasikan dengan **queue**. BFT banyak digunakan dalam **algoritma pencarian jalur terpendek**, **perhitungan tinggi pohon**, dan **visualisasi struktur pohon**.



Pentingnya Tree Traversal

Pentingnya Tree Traversal dalam Struktur Data:

1. **Pemrosesan Data yang Sistematis**: Memberikan cara yang terstruktur untuk mengakses dan memanipulasi setiap node.

2. **Dasar untuk Algoritma Lanjutan**: Traversal menjadi fondasi bagi algoritma pada binary search tree, AVL tree, parsing ekspresi matematika, dan pathfinding.

3. **Optimasi Kinerja**: Pemilihan metode traversal yang tepat dapat meningkatkan efisiensi pencarian, penyimpanan, dan manipulasi data dalam berbagai aplikasi.

Aplikasi Tree Traversal

1. **Struktur Data dan Sistem Berkas**: Traversal tree digunakan untuk menjelajahi struktur hierarkis folder dan file; pre-order traversal menampilkan urutan pembuatan, sedangkan post-order traversal berguna penghapusan rekursif.

2. **Pengindeksan dan Pencarian Basis Data**: Struktur pohon seperti B-Tree dan B+ Tree memanfaatkan traversal untuk pencarian dan manipulasi data; traversal in-order memastikan pengambilan data secara terurut untuk mempercepat query.

Aplikasi Tree Traversal

3. **Kompilasi dan Parsing Ekspresi**: Abstract Syntax Tree (AST) merepresentasikan struktur sintaks program; post-order traversal mengevaluasi ekspresi aritmatika, sementara pre-order traversal digunakan menghasilkan kode mesin.

4. **Artificial Intelligence dan Game Development**: Pohon pencarian dan pohon keputusan memanfaatkan traversal seperti DFS dan BFS untuk mengeksplorasi langkah permainan, perencanaan jalur, serta pengambilan keputusan dalam algoritma AI.

Aplikasi Tree Traversal

5. **Kompresi dan Pengolahan Data**: Algoritma Huffman Coding menggunakan traversal tree untuk menghasilkan kode biner optimal; pre-order traversal membangun tabel kode, sedangkan post-order traversal digunakan untuk dekompresi.

6. **Jaringan Komputer dan Routing**: Traversal pada spanning tree memastikan distribusi data tanpa loop, sementara algoritma Kruskal dan Prim memanfaatkan tree traversal untuk membentuk jalur jaringan yang optimal.

7. **Ilmu Data dan Machine Learning**: Traversal digunakan dalam Decision Tree dan Random Forest untuk menelusuri node dari akar hingga daun, sehingga memungkinkan proses klasifikasi dan prediksi secara sistematis berdasarkan kondisi pada setiap node.

17 November 2026

Sekian

Terima Kasih



Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta